# DROIDDESK

Local Android Development Companion for Windows

## Project Specification & Development Plan

Python · PySide6 · Androguard

Version 1.0 · Technical Design Document

# Table of Contents

Table of Contents ..... 2
1. Executive Summary ..... 4
1.1 Problem Statement ..... 4
1.2 Solution ..... 4
1.3 Design Principles ..... 4
1.4 Positioning ..... 4
2. Technology Stack ..... 6
2.1 Core Platform ..... 6
2.2 Android and Analysis Libraries ..... 6
2.3 Development Tooling ..... 7
3. External Tool Dependencies ..... 8
3.1 Discovery Strategy ..... 8
4. System Architecture ..... 9
4.1 Layers ..... 9
4.2 Threading Model ..... 9
Long-lived external processes — QProcess ..... 9
CPU-bound batch work — QThreadPool ..... 10
4.3 Data Flow — APK Test Run ..... 10
5. Project Structure ..... 12
6. Module Specifications ..... 14
6.1 Device Manager ..... 14
6.2 APK Analyzer ..... 14
6.3 APK Tester and Test Profiles ..... 15
6.4 Logcat Viewer ..... 15
6.5 Secret Scanner ..... 16
Android-specific rules ..... 16
Suppression ..... 16
6.6 APK Privacy Scanner ..... 17
6.7 Permission Analyzer ..... 17
6.8 Environment Manager ..... 17
6.9 Safe Command Runner ..... 18
6.10 Release Gate ..... 18
6.11 Network Insight ..... 19
6.12 Smart File Organizer ..... 19
Duplicate detection ..... 20
Undo journal ..... 20
6.13 Reports ..... 20

7. Data Model ..... 21
8. Critical Implementation Notes ..... 23
8.1 Subprocess Handling on Windows ..... 23
8.2 Scanner Engine Performance ..... 23
8.3 Entropy Heuristics ..... 23
8.4 Qt Model Updates Under Load ..... 24
8.5 APK Analysis Caching ..... 24
8.6 Handling Android App Bundles ..... 24
9. Development Roadmap ..... 25
Phase 1 — Foundation and Device Control ..... 25
Phase 2 — APK Analysis and Logcat ..... 25
Phase 3 — Automated Testing ..... 25
Phase 4 — Security Scanning ..... 25
Phase 5 — Environment and Build Control ..... 26
Phase 6 — File Organizer and Release ..... 26
10. Packaging and Distribution ..... 27
10.1 Build ..... 27
10.2 Installer ..... 27
10.3 Updates ..... 27
11. Testing Strategy ..... 28
Fixture strategy for the scanner ..... 28
Device integration tests ..... 28
12. Risks and Limitations ..... 29
12.1 Statement of Limitations ..... 29
Appendix A — Command Reference ..... 30
Device operations ..... 30
Application lifecycle ..... 30
Capture ..... 30
APK inspection ..... 30
Scanning ..... 30
Appendix B — Getting Started ..... 30

# 1. Executive Summary

DroidDesk is a local desktop application for Windows that consolidates the repetitive, error-prone parts of Android development into a single controlled workspace. It does four things that currently require juggling a terminal, a file explorer, and several one-off scripts: it organizes development artifacts, it installs and exercises APKs on real devices, it inspects what an APK actually exposes, and it blocks builds that are configured against the wrong environment.

The application runs entirely on the developer's machine. It transmits nothing, requires no account, and stores all state in a local SQLite database. Every Android operation is performed by delegating to the official Android SDK command-line tools, which means DroidDesk inherits their correctness rather than reimplementing it.

## 1.1 Problem Statement

Android release workflows fail in predictable ways. A release APK gets built while the project is still pointed at a development Firebase project. A service-account JSON file ends up bundled in the assets directory. A screen recording made for a client demo shows a staging API host in the logs. A developer tests a new build against production data because the device was still configured from last week.

None of these are hard problems individually. They are all easy to catch if someone checks. They keep happening because nobody checks consistently, and because the checking is spread across five different tools with five different output formats.

## 1.2 Solution

DroidDesk provides a single pre-release gate that runs every check at once and produces one verdict. Around that gate sit the supporting tools needed to make the checks meaningful: a device manager, an APK analyzer, an automated test runner, a secret scanner, and an environment switcher.

## 1.3 Design Principles

- **Delegate, do not reimplement.** Manifest parsing, signature verification, and device control are solved problems owned by the Android SDK. DroidDesk orchestrates those tools and presents their output coherently.
- **Classify, do not alarm.** A Google Maps API key inside an APK is expected and correct. A service-account private key is a serious incident. A tool that reports both as "secret found" trains its user to ignore it.
- **Never block the interface.** Scanning a large project or hashing a Downloads folder can take minutes. All long-running work happens off the UI thread with visible, cancellable progress.
- **Every destructive action is reversible.** File organization writes an undo journal before it moves anything.
- **Honest about limits.** Where a check cannot be performed reliably, the application says so rather than producing a green tick that means nothing.

## 1.4 Positioning

**DroidDesk is:** a local developer tool that speeds up Android testing and detects accidental exposure of sensitive configuration before release.

**DroidDesk is not:** a security product, a penetration-testing suite, or a guarantee that an application is safe. It detects common, accidental mistakes. It does not detect deliberate obfuscation, and it makes no claim to.

> **Why this matters:** Positioning the tool as a "security scanner" invites a standard of proof the project cannot meet and creates liability if something slips through. Positioning it as an accident-prevention tool is accurate, defensible, and leaves room to add sophisticated analysis later.

# 2. Technology Stack

The stack is chosen around one constraint: the application spends most of its life spawning external processes, reading their output, and rendering dense tabular data. Python with Qt handles both well, and the Android analysis ecosystem in Python is unusually strong.

## 2.1 Core Platform

| Layer | Technology | Rationale |
|---|---|---|
| Language | Python 3.12+ | Pattern matching, improved error messages, and meaningful performance gains over 3.10 for the file-scanning workloads. |
| UI toolkit | PySide6 (Qt 6.7+) | Official Qt for Python bindings under the LGPL, which permits commercial distribution without a paid licence. Provides native widgets, virtualized tables, and a mature process API. |
| UI pattern | Model/View with QAbstractTableModel | Required for logcat and file listings. A naive widget-per-row approach collapses past a few thousand entries; Qt's model/view renders only visible rows. |
| Process control | QProcess | Asynchronous, signal-driven stdout and stderr streaming that integrates with the Qt event loop without threads or polling. |
| Concurrency | QThreadPool + QRunnable | CPU-bound work (hashing, regex scanning) runs on worker threads; results return to the UI thread via signals. |
| Data validation | Pydantic v2 | Test profiles, environment definitions, and scanner rules are user-editable structured data. Pydantic gives schema validation with readable errors. |
| Persistence | SQLite via sqlite3 + SQLAlchemy Core | Zero-configuration embedded storage. SQLAlchemy Core provides safe query construction without full ORM overhead. |
| Logging | loguru | Structured logging with rotation, configured once rather than per module. |

## 2.2 Android and Analysis Libraries

| Package | Purpose | Notes |
|---|---|---|
| androguard | APK parsing: manifest, permissions, certificates, DEX analysis, resource extraction | Pure Python. Removes the need to parse aapt2 text output for most operations. The single most valuable dependency in this project. |
| pyaxmlparser | Fast, lightweight manifest-only parsing | Use when only package name, version, and SDK levels are needed. Substantially faster than loading full androguard analysis. |
| asn1crypto / cryptography | Reading the signing certificate from META-INF | Backs the signature panel and certificate fingerprint display. |
| xxhash | Non-cryptographic hashing for duplicate detection | Roughly an order of magnitude faster than SHA-256 and entirely sufficient for identifying identical files. |

| Package | Purpose | Notes |
|---|---|---|
| watchdog | Filesystem change notifications | Powers live folder monitoring in the Smart Organizer. |
| send2trash | Moving files to the Recycle Bin instead of deleting | Provides a safety net independent of the application's own undo journal. |
| platformdirs | Correct per-user config and cache locations | Resolves to %APPDATA%\DroidDesk and %LOCALAPPDATA%\DroidDesk on Windows. |
| python-dotenv | Parsing .env files for the environment manager | Reads without executing. |
| regex | Scanner engine pattern matching | Supports possessive quantifiers and atomic groups, which prevent catastrophic backtracking on hostile input. |
| Jinja2 | HTML report templating | Report templates render to HTML, which is then converted to PDF. |
| WeasyPrint | HTML to PDF conversion | Produces higher-quality output than hand-assembling a PDF and lets one template serve both formats. ReportLab is the fallback if WeasyPrint's GTK dependency proves awkward to bundle. |

## 2.3 Development Tooling

| Tool | Purpose |
|---|---|
| uv | Dependency resolution and virtual environment management. Substantially faster than pip and produces a lockfile. |
| ruff | Linting and formatting in a single tool, replacing flake8, isort, and black. |
| mypy (strict) | Static type checking. Worth enforcing strictly given the volume of subprocess output parsing, where types are easy to get wrong. |
| pytest + pytest-qt | Unit and widget testing. pytest-qt drives Qt signals and widgets from tests. |
| pytest-cov | Coverage measurement, targeted at the scanner and parser layers rather than the UI. |
| pre-commit | Runs ruff and mypy before each commit. |
| Nuitka | Compiles to a native Windows executable. Preferred over PyInstaller for this project because it produces faster startup and is harder to trivially unpack. |
| Inno Setup | Windows installer generation, including SDK path detection during setup. |

# 3. External Tool Dependencies

These are separate executables that DroidDesk locates and invokes. They are not bundled into the installer, with the exception of gitleaks. The application detects them on first run and guides the user through pointing at an Android SDK installation if one is not found.

| Tool | Used for | Source |
|---|---|---|
| `adb` | Device discovery, install, uninstall, launch, logcat, screenshot, screen record, file transfer | Android SDK Platform-Tools |
| `aapt2` | Fallback manifest and badging extraction when androguard cannot parse an APK | Android SDK Build-Tools |
| `apksigner` | Signature scheme verification and certificate printing | Android SDK Build-Tools |
| `apkanalyzer` | Size breakdown by file and by DEX package; APK-to-APK comparison | Android SDK cmdline-tools |
| `bundletool` | Generating a universal APK from an .aab so bundles can be tested | Standalone JAR from Google |
| `gitleaks` | Secret detection across project source and extracted APK contents | Single Go binary; bundle with the installer |
| `scrcpy` | Screen mirroring and recording with audio, beyond adb's three-minute limit | Optional; standalone binary |
| `git` | Working-tree cleanliness check in the Release Gate | Optional; already present on most developer machines |

### 3.1 Discovery Strategy

On startup, resolve each tool in the following order and cache the result. Re-verify on cache miss or version change.

1. Explicit path stored in the user's configuration file.
2. ANDROID_HOME or ANDROID_SDK_ROOT environment variable, walking into platform-tools, build-tools (highest version), and cmdline-tools.
3. Default Windows install location: %LOCALAPPDATA%\Android\Sdk.
4. System PATH.
5. If unresolved, present a blocking setup dialog with a folder picker and a link to the platform-tools download.

> **Implementation note:** Never let a missing tool surface as a raw FileNotFoundError traceback. Every feature that depends on an external binary should check availability first and disable its own UI with an explanatory message.

# 4. System Architecture

DroidDesk uses a four-layer architecture. The rule that matters most is that no layer above infrastructure ever calls subprocess directly, and no layer below presentation ever imports a Qt widget.

## 4.1 Layers

| Layer | Contents | Constraint |
|---|---|---|
| Presentation | Qt widgets, views, dialogs, table models | Contains no business logic. Reacts to service signals and forwards user intent. |
| Services | DeviceService, ApkService, ScanService, BuildService, OrganizerService, ReportService | Owns workflow orchestration. Emits Qt signals for progress and completion. Knows nothing about widgets. |
| Domain | Dataclasses and Pydantic models: Device, ApkInfo, Finding, TestProfile, Environment, ReleaseCheck | Pure data and validation. No I/O whatsoever, which makes this layer trivially testable. |
| Infrastructure | ProcessRunner, AdbClient, ApkReader, ScannerEngine, Repository, ToolLocator | The only layer permitted to touch the filesystem, spawn processes, or open the database. |

## 4.2 Threading Model

Qt requires that all widget interaction happen on the main thread. DroidDesk observes this strictly, with two distinct mechanisms for off-thread work depending on the nature of the task.

### Long-lived external processes — QProcess

adb logcat runs indefinitely and produces output continuously. QProcess handles this natively: it starts the process, emits readyReadStandardOutput as data arrives, and integrates with the Qt event loop without any thread being created. No threading is required for I/O-bound subprocess work.

```
class AdbStream(QObject):
    line_received = Signal(str)
    finished = Signal(int)

    def __init__(self, adb_path: Path, args: list[str]) -> None:
        super().__init__()
        self._buffer = ""
        self._proc = QProcess(self)
        self._proc.setProgram(str(adb_path))
        self._proc.setArguments(args)
        self._proc.readyReadStandardOutput.connect(self._on_output)
        self._proc.finished.connect(lambda code, _: self.finished.emit(code))

    def start(self) -> None:
        self._proc.start()

    def stop(self) -> None:
        self._proc.terminate()
        if not self._proc.waitForFinished(2000):
            self._proc.kill()

    def _on_output(self) -> None:
        raw = bytes(self._proc.readAllStandardOutput())
        self._buffer += raw.decode("utf-8", errors="replace")
```

```
        # emit only complete lines; retain any partial trailing line
        *complete, self._buffer = self._buffer.split("\n")
        for line in complete:
            self.line_received.emit(line.rstrip("\r"))
```

### CPU-bound batch work — QThreadPool

Hashing forty thousand files or running two hundred regex patterns across a source tree is genuinely CPU-bound. These run as QRunnable tasks on a QThreadPool, reporting progress through signals. Because CPython's global interpreter lock limits pure-Python parallelism, the scanner uses ProcessPoolExecutor for the regex phase where the workload justifies the serialization cost.

```
class HashTask(QRunnable):
    def __init__(self, paths: list[Path], signals: TaskSignals) -> None:
        super().__init__()
        self.paths = paths
        self.signals = signals
        self._cancelled = False

    def cancel(self) -> None:
        self._cancelled = True

    def run(self) -> None:
        results: dict[str, list[Path]] = defaultdict(list)
        for i, path in enumerate(self.paths):
            if self._cancelled:
                self.signals.cancelled.emit()
                return
            results[hash_file(path)].append(path)
            if i % 200 == 0:
                self.signals.progress.emit(i, len(self.paths))
        self.signals.finished.emit(dict(results))
```

> **Cancellation:** Every long task must be cancellable. Check a cancellation flag inside the loop rather than attempting to kill the thread. A user who starts a scan on the wrong directory should not have to wait four minutes or force-quit the application.

## 4.3 Data Flow — APK Test Run

The following sequence illustrates how the layers cooperate during an automated test, which is the most complex workflow in the application.

```
  User clicks Run Test
        │
        ▼
  TestProfileView ──emits──▶ ApkService.run_test(profile)
        │                                │
        │                                ▼
        │                     ApkReader.inspect(apk)      ← androguard, in-process
        │                                │
        │                                ▼
        │                     AdbClient.install(device, apk)
        │                                │
        │                                ▼
        │                     AdbClient.launch(device, package)
        │                                │
        │                          ┌─────┴──────┐
        │                          ▼            ▼
        │              logcat stream        screenshot capture
        │                          │            │
        │                          └─────┬──────┘
        │                                ▼
        │                     CrashDetector.evaluate(lines)
        │                                │
```


AdbClient.uninstall(device, package)
signals: step_started, step_finished, test_complete
Repository.save_run(result)
ReportService.render(result)

## 5. Project Structure

The layout below mirrors the architecture. Package boundaries are enforced by import discipline and checked in review; a module under core/ that imports from ui/ is a defect.

```
droiddesk/
├── pyproject.toml              # uv / hatchling project definition
├── uv.lock
├── README.md
├── .pre-commit-config.yaml
│
├── src/droiddesk/
│   ├── __main__.py             # entry point, QApplication bootstrap
│   ├── app.py                  # DI wiring, service construction
│   ├── config.py               # settings model, paths, persistence
│   │
│   ├── core/                   # domain layer — no I/O, no Qt
│   │   ├── models.py           # Device, ApkInfo, Finding, TestRun
│   │   ├── profiles.py         # TestProfile, Environment, CommandProfile
│   │   ├── severity.py         # Finding classification rules
│   │   └── errors.py           # exception hierarchy
│   │
│   ├── infra/                  # infrastructure layer
│   │   ├── tools.py            # ToolLocator: find adb, aapt2, gitleaks
│   │   ├── process.py          # ProcessRunner, AdbStream
│   │   ├── adb.py              # AdbClient: typed wrapper over adb
│   │   ├── apk_reader.py       # androguard facade
│   │   ├── scanner/
│   │   │   ├── engine.py       # rule loading, matching, dedup
│   │   │   ├── rules.toml      # gitleaks-compatible rule definitions
│   │   │   ├── android.toml    # Android-specific rules
│   │   │   └── entropy.py      # Shannon entropy scoring
│   │   ├── hashing.py          # three-tier duplicate detection
│   │   └── db.py               # SQLite schema, migrations, Repository
│   │
│   ├── services/               # application layer
│   │   ├── device_service.py
│   │   ├── apk_service.py
│   │   ├── test_service.py     # automated test orchestration
│   │   ├── scan_service.py
│   │   ├── env_service.py      # environment switching
│   │   ├── build_service.py    # safe command runner
│   │   ├── gate_service.py     # release gate evaluation
│   │   ├── organizer_service.py
│   │   └── report_service.py
│   │
│   ├── ui/                     # presentation layer
│   │   ├── main_window.py      # shell, sidebar, stacked pages
│   │   ├── pages/
│   │   │   ├── dashboard.py
│   │   │   ├── devices.py
│   │   │   ├── apk_tester.py
│   │   │   ├── logcat.py
│   │   │   ├── analyzer.py
│   │   │   ├── security.py
│   │   │   ├── organizer.py
│   │   │   ├── commands.py
│   │   │   └── reports.py
│   │   ├── models/             # QAbstractTableModel subclasses
│   │   │   ├── logcat_model.py
│   │   │   ├── findings_model.py
│   │   │   └── files_model.py
│   │   ├── widgets/            # reusable components
│   │   │   ├── device_selector.py
│   │   │   ├── severity_badge.py
│   │   │   ├── progress_panel.py
│   │   │   └── code_view.py
│   │   └── theme.py            # QSS stylesheet, palette, fonts
```

```
│   │
│   ├── reports/
│   │   ├── templates/          # Jinja2: test_report.html, gate.html
│   │   └── renderer.py         # HTML → PDF via WeasyPrint
│   │
│   └── resources/              # icons, fonts, QSS, bundled gitleaks
│
├── tests/
│   ├── unit/                   # core/ and infra/ — the bulk of coverage
│   ├── integration/            # against a real emulator, marked slow
│   ├── fixtures/               # sample APKs, synthetic secret files
│   └── conftest.py
│
└── packaging/
    ├── nuitka_build.py
    └── installer.iss           # Inno Setup script
```

# 6. Module Specifications

Each module below is specified with its responsibility, its inputs, the mechanism it uses, and what it produces. Modules are listed in build order, not menu order.

## 6.1 Device Manager

**Responsibility.** Discover connected devices and emulators, expose their properties, and provide the device selection that every other Android feature depends on.

**Mechanism.** Poll adb devices -l every two seconds while the Devices page is visible; fall back to five seconds when it is not. Enrich each entry with adb -s <serial> shell getprop for model, Android version, API level, ABI, and screen density.

- Handle the unauthorized state explicitly — show the pairing instruction, not an error.
- Handle the offline state, which usually means a stalled USB connection; offer an adb reconnect action.
- Support wireless debugging via adb pair and adb connect with a host:port entry field.
- Mark a device as a Test Device Profile, which attaches an allowed-environment policy used by the Release Gate and the test runner.

**Output.** A list of Device objects, cached in memory and republished on change via a devices_changed signal.

## 6.2 APK Analyzer

**Responsibility.** Read an APK and present everything derivable from it without installing it.

**Mechanism.** androguard's APK class for manifest, permissions, activities, services, and certificates. apkanalyzer for size breakdown, which androguard does not provide in a comparable form. apksigner for signature scheme verification.

| Panel | Data shown | Source |
|---|---|---|
| Identity | Package name, version name, version code, min/target/compile SDK | androguard manifest |
| Signature | Signature schemes present (v1–v4), certificate subject, SHA-256 fingerprint, validity dates | apksigner + cryptography |
| Permissions | Declared permissions with protection level and plain-language explanation | androguard + local permission catalogue |
| Components | Activities, services, receivers, providers, and which are exported | androguard manifest |
| Size | Total size, download size estimate, breakdown by DEX / resources / assets / native libs | apkanalyzer apk file-size |
| Native libs | ABIs present and per-ABI size | ZIP directory listing of lib/ |
| Compare | Two APKs side by side: size delta, permission delta, version delta | apkanalyzer apk compare + manifest diff |

> **Performance:** Full androguard analysis on a large APK can take ten seconds or more because it disassembles the DEX. Use pyaxmlparser for the fast identity panel, load full analysis lazily only when the user opens a panel that needs it, and cache results keyed by the APK's file hash.

## 6.3 APK Tester and Test Profiles

**Responsibility.** Run a reproducible sequence of install, launch, observe, capture, and clean up against a selected device.

**Mechanism.** A state machine over adb operations, driven by a TestProfile that defines which steps are enabled.

| Step | Command | Pass condition |
|---|---|---|
| Validate | `androguard, in-process` | Manifest parses; package name and version extracted |
| Install | `adb -s S install -r -t <apk>` | Output contains Success |
| Launch | `adb -s S shell monkey -p PKG -c android.intent.category.LAUNCHER 1` | Process appears in pidof within timeout |
| Observe | `adb -s S logcat --pid=<pid> -v threadtime` | No FATAL EXCEPTION, no ANR, no Process died |
| Screenshot | `adb -s S exec-out screencap -p` | Non-empty PNG returned |
| Permissions | `adb -s S shell dumpsys package PKG` | Runtime grants recorded for the report |
| Network | `Static host extraction from the APK` | Hosts listed for review; informational only |
| Uninstall | `adb -s S uninstall PKG` | Output contains Success |

**Crash detection.** Do not scan for the word "error", which appears constantly in normal Android logs. Match specifically on FATAL EXCEPTION, Process <package> has died, ANR in <package>, and signal 11 (SIGSEGV) lines scoped to the target PID. Everything else is classified as a warning and surfaced separately.

**Output.** A TestRun record with per-step status, duration, captured log excerpt, screenshot path, and an overall verdict of PASS, PASS WITH WARNINGS, or FAIL.

## 6.4 Logcat Viewer

**Responsibility.** Stream, filter, and search device logs without freezing.

**Mechanism.** AdbStream feeds a ring-buffer-backed QAbstractTableModel with a fixed capacity, defaulting to fifty thousand lines. Filtering is applied through QSortFilterProxyModel over tag, priority level, PID, and a free-text regular expression.

- Parse the threadtime format into structured columns rather than displaying raw text; this makes filtering and colouring possible.
- Batch model updates. Emitting a signal per line will saturate the event loop; buffer incoming lines and flush every 100 ms.

- Provide a pause control that stops appending without killing the underlying process, so the buffer is preserved.
- Support export of the current filtered view to a text file for attaching to bug reports.

## 6.5 Secret Scanner

**Responsibility.** Find credentials in a project tree before they reach a commit or a build.

**Mechanism.** A two-stage engine. Stage one shells out to the bundled gitleaks binary with --report-format json, which supplies roughly two hundred maintained rules for common providers. Stage two applies DroidDesk's own Android-specific rules and entropy heuristics. Results from both stages are merged and deduplicated by file, line, and rule.

### Android-specific rules

| Pattern | Classification | Reasoning |
|---|---|---|
| service-account.json with private_key field | Critical | A Google Cloud service account key grants server-side access. Never belongs in a mobile project. |
| *.jks, *.keystore, *.p12 tracked in the repository | Critical | Signing keys. Compromise permits publishing forged updates. |
| AWS access key ID and secret pair | Critical | Direct infrastructure access. |
| Private key PEM blocks | Critical | Any BEGIN PRIVATE KEY block in a client project. |
| .env files containing DATABASE_URL or SECRET_KEY | High | Backend credentials with no legitimate reason to exist in a client build. |
| Firebase database secret (legacy token) | High | Deprecated but still grants full database access where present. |
| google-services.json | Informational | Expected in every Firebase-enabled Android project. Flag its presence inside a built APK only if it contains an unexpected field. |
| Google Maps or Places API key in resources | Needs review | Designed to ship in the APK. The finding should recommend restricting the key by package name and SHA-1 fingerprint, not removing it. |
| High-entropy string over 32 characters | Needs review | Heuristic only. Produces false positives on hashes, UUIDs, and minified assets; must be reviewable and suppressible. |

> **The classification is the product.** A scanner that reports every match at the same severity is worse than no scanner, because it trains the developer to dismiss the panel. The Maps-key case is the clearest example: the correct advice is "restrict this key," not "remove this key." Getting this distinction right is what separates DroidDesk from a wrapper around grep.

### Suppression

Findings can be suppressed with a reason, stored in a .droiddesk/suppressions.toml file inside the project so the decision travels with the repository and is visible in code review. Suppressions record the rule ID,

the file, a content hash of the matched line, and the justification. If the line changes, the suppression lapses and the finding returns.

### 6.6 APK Privacy Scanner

**Responsibility.** Inspect a built APK for configuration that should not have shipped.

**Mechanism.** An APK is a ZIP file. Enumerate its entries, extract the ones worth inspecting to a temporary directory, and run the scanner engine over them. In addition, extract string resources from resources.arsc via androguard, since API keys most often live there rather than in a loose file.

- Check for the presence of .env, service-account.json, *.pem, *.jks, and *.keystore anywhere in the archive — these should never be present and are always critical.
- Scan assets/ and res/raw/ thoroughly; these are the directories where files get bundled by accident.
- Extract all string resources and run credential rules across them.
- Extract string constants from the DEX using androguard's analysis to catch keys hardcoded in source rather than resources. This is slower; make it an opt-in deep scan.
- Report the archive path of every finding so the developer can trace it back to a source directory.

### 6.7 Permission Analyzer

**Responsibility.** Explain what an APK is asking for in language that supports a release decision.

Maintain a local catalogue mapping each Android permission to its protection level, a plain-language description, and a review flag. Permissions warranting explicit review include ACCESS_BACKGROUND_LOCATION, READ_SMS, RECEIVE_SMS, READ_CONTACTS, RECORD_AUDIO, QUERY_ALL_PACKAGES, REQUEST_INSTALL_PACKAGES, and SYSTEM_ALERT_WINDOW — all of which either trigger Play Store policy review or commonly appear through a transitive dependency the developer did not intend to include.

Where possible, attribute a permission to the library that declared it by inspecting merged manifest artifacts in the project's build directory. Knowing that ACCESS_BACKGROUND_LOCATION arrived through an analytics SDK is considerably more actionable than knowing it is present.

### 6.8 Environment Manager

**Responsibility.** Make the active environment visible and switching deliberate.

**Mechanism.** Environments are declared in .droiddesk/environments.toml, each naming an env file, a Firebase configuration file, a flavour, and an expected API host. Switching copies the selected environment's files into place, after backing up what was there.

```
[environments.dev]
label       = "Development"
env_file    = ".env.dev"
firebase    = "config/google-services.dev.json"
flavor      = "dev"
api_host    = "dev-api.example.com"
allow_release_build = false

[environments.production]
label       = "Production"
env_file    = ".env.production"
firebase    = "config/google-services.prod.json"
flavor      = "prod"
api_host    = "api.example.com"
```

```
allow_release_build = true
require_clean_git   = true
```

The active environment is displayed persistently in the application's status bar, coloured by risk. This single piece of always-visible state prevents a substantial fraction of the mistakes the tool exists to catch.

## 6.9 Safe Command Runner

**Responsibility.** Intercept build commands and validate their preconditions before execution.

Command profiles are stored per project with attached guard rules. When a command is invoked, the runner evaluates every guard and presents the result. Guards that fail produce either a warning that can be overridden with an explicit acknowledgement, or a hard block.

| Guard | Checks | Failure mode |
|---|---|---|
| `environment_matches` | Active environment equals the profile's required environment | Hard block |
| `release_allowed` | Active environment permits release builds | Hard block |
| `git_clean` | git status --porcelain returns no output | Warning, overridable |
| `secret_scan_clean` | No critical or high findings in the project | Hard block on critical, warning on high |
| `no_debug_flags` | No debuggable=true in the release manifest | Hard block |
| `version_bumped` | versionCode exceeds the last recorded release build | Warning, overridable |

Command output streams into an embedded terminal view built on QProcess with the same line-buffered handling as logcat, with ANSI colour codes parsed into text formatting.

## 6.10 Release Gate

**Responsibility.** Compose every other check into a single go or no-go decision.

The gate runs its checks in dependency order, short-circuiting on any hard failure. It produces a score out of one hundred alongside a categorical verdict; the score is for tracking improvement over time, while the verdict is what governs the build.

| Category | Check | Weight | Blocking |
|---|---|---|---|
| Project | Package name matches the expected release identifier | 10 | Yes |
| Environment | Production environment active and release-permitted | 20 | Yes |
| Git | Working tree clean, on a release branch | 10 | No |
| Secrets | No critical findings in the project tree | 20 | Yes |
| APK | Valid, signed with v2 or later, correct target SDK | 15 | Yes |
| Privacy | No credential files present inside the APK | 15 | Yes |
| Permissions | No unreviewed high-risk permissions | 5 | No |

| Category | Check | Weight | Blocking |
|---|---|---|---|
| Size | Under the configured threshold | 5 | No |

> **Override policy:** Blocking checks can be overridden only through a dialog that requires typing the reason, and the override is recorded permanently in the build history with the reason attached. Making the override possible but visible is more effective than making it impossible, which simply drives people to bypass the tool entirely.

## 6.11 Network Insight

**Responsibility.** Show which external services an application contacts.

> **Read this before designing the feature.** HTTPS traffic from a release build cannot be intercepted. Since Android 7.0, applications trust only system certificate authorities unless their network security configuration explicitly opts into user-installed ones — and release builds do not. Any design that assumes a proxy will see decrypted traffic will fail on exactly the builds that matter most.

DroidDesk therefore implements two separate capabilities and labels them clearly.

**Static host extraction (always available).** Extract every URL and hostname from the APK's string resources, DEX string constants, and network security configuration. This produces a definitive list of hosts the application contains references to. It cannot show which were actually contacted, but it works on any APK, requires no device, and cannot be evaded by an application choosing not to make a request during the test window.

**Live connection observation (debug builds only).** For debug builds, offer optional mitmproxy integration with adb reverse and a generated CA certificate. Detect whether the target APK is debuggable and whether its network security configuration trusts user certificates; if not, disable the feature with an explanation rather than silently capturing nothing.

The static approach delivers most of the practical value. A developer asking "does this build still point at staging" gets a correct answer from string extraction alone.

## 6.12 Smart File Organizer

**Responsibility.** Classify and tidy development artifacts, with full reversibility.

**Mechanism.** A rule engine evaluating each file against ordered rules; the first match wins. Rules combine extension, filename pattern, magic bytes, size, and age. Content sniffing distinguishes a .zip that is actually a Flutter project archive from an ordinary archive.

```
[[rules]]
name        = "Release APKs"
match_ext   = [".apk", ".aab"]
match_name  = ".*-(release|prod).*"
destination = "APK/Release"
priority    = 10

[[rules]]
name        = "Screenshots"
match_ext   = [".png", ".jpg"]
match_name  = "^(Screenshot|Screen Shot|IMG_).*"
destination = "Images/Screenshots"
priority    = 20
```

```
[[rules]]
name        = "Stale archives"
match_ext   = [".zip", ".tar.gz", ".7z"]
older_than_days = 90
destination = "Archives/Old"
priority    = 30
```

### Duplicate detection

Hashing every file in a large Downloads folder is prohibitively slow and unnecessary. Use three tiers, each eliminating candidates before the next runs.

6. Group by exact file size. Files of differing size cannot be identical; this eliminates the overwhelming majority of comparisons at near-zero cost.
7. Within each size group of more than one file, hash the first and last 8 KB. This catches differing files cheaply without reading them fully.
8. Only for files still colliding after tier two, compute a full xxhash of the contents.

On a typical Downloads directory this reduces full-file reads by well over ninety percent.

### Undo journal

Before any batch operation, write a journal to the application data directory recording every intended move as a source and destination pair, with a batch identifier and timestamp. Perform the moves, updating the journal as each completes. Undo replays the journal in reverse. If the application is killed mid-operation, the journal is detected on next launch and the user is offered a rollback.

Deletions never call os.remove. They call send2trash, so the operating system's own recovery mechanism remains available as a second layer of protection.

## 6.13 Reports

One Jinja2 template set renders every report format. HTML is the source of truth; PDF is produced from it with WeasyPrint, and JSON is serialized directly from the domain model for machine consumption in continuous integration.

| Format | Use case |
|---|---|
| HTML | Interactive review, opened in the default browser, with collapsible finding details |
| PDF | Archival, client delivery, attaching to a release ticket |
| JSON | Machine consumption; a CI step can read the gate verdict and fail the pipeline |
| Markdown | Pasting into a pull request description or issue tracker |

## 7. Data Model

All persistent state lives in a single SQLite database at %LOCALAPPDATA%\DroidDesk\droiddesk.db. Schema changes are handled by numbered migration scripts applied in order at startup, with the current version tracked in a schema_version table.

```
CREATE TABLE project (
    id            INTEGER PRIMARY KEY,
    name          TEXT NOT NULL,
    root_path     TEXT NOT NULL UNIQUE,
    package_name  TEXT,
    created_at    TEXT NOT NULL,
    last_opened   TEXT
);

CREATE TABLE apk (
    id            INTEGER PRIMARY KEY,
    file_path     TEXT NOT NULL,
    file_hash     TEXT NOT NULL,          -- xxhash, used as analysis cache key
    package_name  TEXT NOT NULL,
    version_name  TEXT,
    version_code  INTEGER,
    target_sdk    INTEGER,
    min_sdk       INTEGER,
    size_bytes    INTEGER,
    analyzed_at   TEXT NOT NULL,
    UNIQUE(file_hash)
);

CREATE TABLE test_run (
    id            INTEGER PRIMARY KEY,
    apk_id        INTEGER REFERENCES apk(id),
    device_serial TEXT NOT NULL,
    profile_name  TEXT NOT NULL,
    started_at    TEXT NOT NULL,
    duration_ms   INTEGER,
    verdict       TEXT NOT NULL,          -- PASS | WARN | FAIL
    steps_json    TEXT NOT NULL,          -- serialized per-step results
    log_excerpt   TEXT,
    screenshot    TEXT                    -- path on disk
);

CREATE TABLE finding (
    id            INTEGER PRIMARY KEY,
    scan_id       INTEGER REFERENCES scan(id) ON DELETE CASCADE,
    rule_id       TEXT NOT NULL,
    severity      TEXT NOT NULL,          -- critical | high | review | info
    file_path     TEXT NOT NULL,
    line_number   INTEGER,
    match_hash    TEXT NOT NULL,          -- hash of matched line, for suppression
    redacted      TEXT NOT NULL,          -- masked preview, never the raw secret
    suppressed    INTEGER DEFAULT 0,
    suppress_note TEXT
);

CREATE TABLE scan (
    id            INTEGER PRIMARY KEY,
    project_id    INTEGER REFERENCES project(id),
    target_type   TEXT NOT NULL,          -- project | apk
    target_path   TEXT NOT NULL,
    started_at    TEXT NOT NULL,
    files_scanned INTEGER,
    duration_ms   INTEGER
);

CREATE TABLE build_record (
```

```
    id            INTEGER PRIMARY KEY,
    project_id    INTEGER REFERENCES project(id),
    command       TEXT NOT NULL,
    environment   TEXT NOT NULL,
    gate_score    INTEGER,
    gate_verdict  TEXT,
    overridden    INTEGER DEFAULT 0,
    override_note TEXT,
    exit_code     INTEGER,
    started_at    TEXT NOT NULL
);

CREATE INDEX idx_finding_scan ON finding(scan_id);
CREATE INDEX idx_test_apk    ON test_run(apk_id);
CREATE INDEX idx_build_proj  ON build_record(project_id, started_at DESC);
```

> **Never store the secret itself.** The finding table stores a redacted preview and a hash of the matched line. Writing discovered credentials into a plaintext SQLite file would make DroidDesk the largest credential exposure in the developer's workflow — the exact failure it exists to prevent.

# 8. Critical Implementation Notes

These are the details that will cost time if discovered late.

## 8.1 Subprocess Handling on Windows

- **Suppress console windows.** Every subprocess launched from a windowed Python application will flash a console window unless creationflags=subprocess.CREATE_NO_WINDOW is passed. With QProcess, set the same flag through setCreateProcessArgumentsModifier. Missing this makes the application look broken.
- **Never use shell=True.** Paths on Windows contain spaces, and project paths contain user-supplied text. Pass arguments as a list.
- **Decode defensively.** Logcat output contains arbitrary application strings and is not guaranteed valid UTF-8. Always decode with errors="replace"; a UnicodeDecodeError that kills the log stream is a difficult bug to diagnose after the fact.
- **Quote long paths.** Windows path length limits still bite on deeply nested Flutter build directories. Prefer pathlib and consider enabling long path support in the installer manifest.
- **Always set timeouts.** An adb command against an unresponsive device will hang indefinitely. Every one-shot call needs a timeout and a clear error.

## 8.2 Scanner Engine Performance

Running two hundred regular expressions over a project tree naively means two hundred passes over every file. Three techniques keep this fast.

9. Pre-filter with Aho-Corasick. Most credential patterns contain a fixed literal prefix — AKIA, ghp_, sk_live_, AIzaSy, and so on. Build a single automaton over these literals and run it once per file. Only files producing a literal hit proceed to full regex evaluation. This eliminates the overwhelming majority of files in one pass.
10. Skip what cannot contain text. Exclude build/, .git/, node_modules/, .gradle/, .dart_tool/, and binary extensions by default. Detect binary content by checking for null bytes in the first 8 KB.
11. Cap file size. Files above a threshold, defaulting to 10 MB, are skipped with a note rather than scanned. Minified bundles and asset blobs produce entropy false positives and consume disproportionate time.

Compile all patterns once at engine construction, never inside the per-file loop. Use the regex module rather than re, and write patterns with atomic groups to prevent catastrophic backtracking on adversarial input.

## 8.3 Entropy Heuristics

Shannon entropy identifies random-looking strings, which correlates with credentials but also with hashes, UUIDs, base64 images, and minified JavaScript. Applied naively it produces enough noise to make the whole feature useless.

- Apply entropy only to strings assigned to a variable whose name suggests a credential — key, token, secret, password, auth, credential.
- Require a minimum length of 32 characters and a threshold of roughly 4.5 bits per character for base64-like alphabets.

- Maintain an exclusion list for known-benign shapes: UUIDs, git SHAs, standard hash lengths, and data URIs.
- Always classify entropy findings as "needs review," never as critical. They are a hint, not a determination.

## 8.4 Qt Model Updates Under Load

A logcat stream can produce thousands of lines per second during application startup. Calling beginInsertRows and endInsertRows per line will make the interface unresponsive.

```
class LogcatModel(QAbstractTableModel):
    def __init__(self, capacity: int = 50_000) -> None:
        super().__init__()
        self._rows: deque[LogLine] = deque(maxlen=capacity)
        self._pending: list[LogLine] = []
        self._timer = QTimer(self)
        self._timer.setInterval(100)
        self._timer.timeout.connect(self._flush)
        self._timer.start()

    def queue(self, line: LogLine) -> None:
        self._pending.append(line)      # cheap; no model signal

    def _flush(self) -> None:
        if not self._pending:
            return
        # deque with maxlen evicts from the front automatically,
        # so a full reset is simpler and correct at this batch size
        self.beginResetModel()
        self._rows.extend(self._pending)
        self._pending.clear()
        self.endResetModel()
```

Batching model signals to a fixed interval decouples interface responsiveness from log volume entirely.

## 8.5 APK Analysis Caching

Analysis is keyed by the APK's content hash, not its path. A developer rebuilding repeatedly produces a new file at the same path each time; a path-keyed cache would serve stale results, which is worse than no cache. Hash the file with xxhash — fast enough to be imperceptible even on a 100 MB APK — and use that as the key.

## 8.6 Handling Android App Bundles

Modern Play Store releases ship .aab rather than .apk. A bundle cannot be installed directly. Detect the extension, and when a bundle is selected, invoke bundletool build-apks with --mode=universal to produce an installable artifact, then proceed with the normal pipeline. Make this transparent but visible: the report should state that testing was performed against a universal APK generated from the bundle, since a universal APK differs from what a device would actually receive from the Play Store.

# 9. Development Roadmap

Six phases, ordered so that each produces something usable on its own. The sequence deliberately front-loads the process-orchestration layer, because every later feature depends on it and its risks surface immediately.

### Phase 1 — Foundation and Device Control

**Target: 2 weeks.** Produces a working device manager and a proven subprocess layer.

- Project scaffolding, uv environment, ruff and mypy configuration, pre-commit hooks.
- ToolLocator with the full discovery chain and a first-run setup dialog.
- ProcessRunner and AdbStream with Windows console suppression and timeout handling.
- AdbClient covering devices, getprop, install, uninstall, and shell.
- Main window shell: sidebar navigation, stacked pages, status bar, QSS theme.
- Devices page with live polling, unauthorized and offline state handling.

**Exit criterion:** connect a device, see it appear with correct properties, install an APK by drag and drop, and see the result. If the process layer is wrong, it will be obvious here rather than in month three.

### Phase 2 — APK Analysis and Logcat

**Target: 2 weeks.** Produces a standalone APK inspector that is already worth using.

- ApkReader facade over androguard and pyaxmlparser, with hash-keyed caching.
- Analyzer page: identity, signature, permissions, components, size breakdown.
- Permission catalogue with plain-language descriptions and review flags.
- Logcat page with the batched model, structured column parsing, and filtering.
- SQLite schema, migration runner, and the apk and project tables.

### Phase 3 — Automated Testing

**Target: 2 weeks.** Delivers the headline feature.

- TestProfile model, editor UI, and persistence.
- Test state machine with per-step status, timing, and cancellation.
- Crash detection scoped to the target PID with precise pattern matching.
- Screenshot capture through exec-out screencap.
- Report rendering: Jinja2 templates, HTML output, PDF via WeasyPrint, JSON export.
- Test history view backed by the test_run table.

### Phase 4 — Security Scanning

**Target: 3 weeks.** The largest phase; the classification work is where the time goes.

- Scanner engine: rule loading, Aho-Corasick pre-filter, compiled regex evaluation, entropy scoring.
- gitleaks integration with JSON parsing and result merging.
- Android-specific rule set and the severity classification table.
- APK privacy scanner: archive enumeration, resource string extraction, optional DEX deep scan.

- Findings view with severity grouping, redacted previews, and suppression workflow.
- Suppression file format and lapse-on-change behaviour.

### Phase 5 — Environment and Build Control

**Target: 2 weeks.** Turns detection into prevention.

- Environment definitions, switching with backup, persistent status bar indicator.
- Command profiles with attached guard rules.
- Safe Command Runner with the guard evaluation pipeline and override dialog.
- Embedded terminal view with ANSI parsing.
- Release Gate composing all checks into a scored verdict.
- Build history with permanent override records.

### Phase 6 — File Organizer and Release

**Target: 2 weeks.** The last feature, plus everything needed to ship.

- Rule engine with content sniffing and TOML rule definitions.
- Three-tier duplicate detection on a worker thread with progress and cancellation.
- Preview-before-apply, undo journal, crash recovery, send2trash integration.
- Large-file and empty-folder reports.
- Nuitka build configuration and Inno Setup installer with SDK detection.
- Documentation, sample rule sets, and a first-run walkthrough.

> **On sequencing:** Resist building the dashboard first. A dashboard is an aggregation of features that do not yet exist, so it will be rebuilt three times. Build it in Phase 6 when there is real data to aggregate.

# 10. Packaging and Distribution

## 10.1 Build

Nuitka compiles the application to a native Windows executable. It is preferred over PyInstaller here for faster startup and because the resulting binary is not trivially unpacked back into source.

```
python -m nuitka ^
  --standalone ^
  --enable-plugin=pyside6 ^
  --windows-console-mode=disable ^
  --windows-icon-from-ico=resources/droiddesk.ico ^
  --include-data-dir=src/droiddesk/resources=droiddesk/resources ^
  --include-data-dir=src/droiddesk/reports/templates=droiddesk/reports/templates ^
  --include-package-data=androguard ^
  --output-dir=dist ^
  src/droiddesk/__main__.py
```

- Expect a 90 to 140 MB distribution directory. PySide6 and androguard are both large; this is normal and acceptable for a developer tool.
- The --windows-console-mode=disable flag is essential. Without it, a console window accompanies the application.
- Androguard loads data files at runtime; --include-package-data is required or it will fail only in the packaged build, not in development.
- WeasyPrint depends on GTK libraries on Windows. Verify PDF export in the packaged build early in Phase 3, not at release. If bundling proves impractical, fall back to ReportLab for PDF and keep WeasyPrint out of the dependency tree.

## 10.2 Installer

Inno Setup produces the installer. It should perform SDK detection during setup rather than deferring the problem to first launch.

- Detect an existing Android SDK and pre-populate the tool paths in the initial configuration.
- If no SDK is found, offer to download platform-tools directly and place it in the application directory.
- Bundle the gitleaks binary; it is a single self-contained executable with no dependencies.
- Register a file association for .apk so double-clicking opens the analyzer.
- Write configuration to %APPDATA%\DroidDesk and the database to %LOCALAPPDATA%\DroidDesk — never into Program Files, which is not user-writable.
- Enable long path support in the application manifest.

## 10.3 Updates

Check a version endpoint on startup, at most once per day, and notify without auto-installing. Developer tools that update themselves without asking are poorly received. Include a full changelog link in the notification.

## 11. Testing Strategy

Coverage effort is concentrated where correctness is hard to verify by inspection: parsers, classifiers, and state machines. The user interface is tested lightly and manually.

| Layer | Approach | Target |
|---|---|---|
| core/ | Pure unit tests. No mocking required, since the layer has no I/O. | 95% coverage |
| infra/scanner | Fixture-driven tests over synthetic files containing known planted secrets, plus a set of known false positives that must not fire. | 90% coverage |
| infra/apk_reader | Tests against a small set of committed sample APKs: a normal build, a bundle-derived universal APK, an unsigned APK, and a deliberately corrupt file. | 85% coverage |
| infra/adb | AdbClient tested against a fake ProcessRunner returning captured real adb output. Verifies parsing without requiring a device. | 85% coverage |
| services/ | Integration tests with fake infrastructure. Verifies orchestration order and signal emission. | 75% coverage |
| ui/ | pytest-qt smoke tests: each page constructs, renders, and handles an empty state without raising. | Smoke only |

### Fixture strategy for the scanner

The scanner's value depends entirely on its precision, so its test suite must include both directions. Maintain two fixture directories: one containing planted credentials of every rule type that must all be detected, and one containing realistic false-positive candidates — UUIDs, git SHAs, base64 images, minified bundles, and legitimate Maps API keys — that must produce either no finding or a correctly classified low-severity one. A rule change that improves recall while quietly destroying precision will be caught immediately.

### Device integration tests

Mark tests requiring a real device or emulator with @pytest.mark.device and exclude them from the default run. Provide a make target that boots a headless emulator, runs the marked suite, and tears it down. These tests are slow and flaky by nature; they belong in a nightly run, not in pre-commit.

# 12. Risks and Limitations

Stated plainly, both for the project's own planning and for the documentation shipped to users.

| Risk | Impact | Mitigation |
|---|---|---|
| HTTPS traffic cannot be inspected on release builds | The most visually appealing feature is the least deliverable | Ship static host extraction as the primary capability; scope live capture to debug builds and label it accordingly |
| Secret scanning produces false positives | Users disable the feature entirely | Rigorous severity classification, a maintained false-positive fixture suite, and a first-class suppression workflow |
| Secret scanning misses obfuscated credentials | False confidence at the moment of release | Documentation states explicitly that the tool detects accidental exposure, not deliberate concealment; the Release Gate never reports "secure," only "no obvious issues found" |
| Androguard is slow on large APKs | Interface appears frozen during analysis | Lazy panel loading, pyaxmlparser for fast paths, hash-keyed caching, all analysis off the UI thread |
| External tool versions drift | Output parsing breaks silently after an SDK update | Record tool versions with every result; parse defensively and surface an explicit "unexpected output format" error rather than returning wrong data |
| Nuitka build differs from the development environment | Failures appear only after packaging | Build and smoke-test the packaged executable in CI from Phase 1 onward, not at release |
| Scope expansion | Six phases become eighteen months | Each phase ships something usable independently; resist starting a phase before the previous one is complete |

## 12.1 Statement of Limitations

This text, or a close variant of it, should appear in the application's about dialog and at the head of every generated report.

> **For inclusion in reports:** DroidDesk performs automated checks for commonly misplaced configuration and credential files. It detects accidental exposure, not deliberate concealment. A clean result indicates that no issues were found by the checks performed; it is not a guarantee that an application is secure, and it does not substitute for security review. All findings should be assessed by a person before acting on them.

# Appendix A — Command Reference

The external commands DroidDesk wraps, for implementation reference.

### Device operations

```
adb devices -l                              # list with product/model/device
adb -s SERIAL shell getprop ro.product.model
adb -s SERIAL shell getprop ro.build.version.release
adb -s SERIAL shell getprop ro.build.version.sdk
adb -s SERIAL shell getprop ro.product.cpu.abi
adb pair HOST:PORT                          # wireless pairing
adb connect HOST:PORT
adb -s SERIAL reconnect                     # recover from offline state
```

### Application lifecycle

```
adb -s SERIAL install -r -t app.apk         # -r replace, -t allow test builds
adb -s SERIAL install-multiple base.apk split.apk
adb -s SERIAL uninstall com.example.app
adb -s SERIAL shell monkey -p com.example.app \
    -c android.intent.category.LAUNCHER 1    # launch main activity
adb -s SERIAL shell pidof com.example.app
adb -s SERIAL shell am force-stop com.example.app
adb -s SERIAL shell pm clear com.example.app
adb -s SERIAL shell dumpsys package com.example.app | findstr granted
```

### Capture

```
adb -s SERIAL logcat -c                     # clear buffer before a test
adb -s SERIAL logcat --pid=PID -v threadtime
adb -s SERIAL exec-out screencap -p > shot.png
adb -s SERIAL shell screenrecord --time-limit 180 /sdcard/rec.mp4
adb -s SERIAL pull /sdcard/rec.mp4 ./rec.mp4
scrcpy -s SERIAL --record=out.mp4           # no time limit, with audio
```

### APK inspection

```
aapt2 dump badging app.apk                  # fallback identity extraction
aapt2 dump permissions app.apk
apksigner verify --verbose --print-certs app.apk
apkanalyzer apk summary app.apk
apkanalyzer apk file-size app.apk
apkanalyzer apk compare old.apk new.apk
apkanalyzer dex packages app.apk
bundletool build-apks --bundle=app.aab --output=app.apks \
    --mode=universal                        # produce an installable APK
```

### Scanning

```
gitleaks detect --source=PROJECT_DIR \
    --report-format=json --report-path=- --no-banner
gitleaks detect --source=EXTRACTED_APK_DIR --no-git \
    --config=android-rules.toml --report-format=json --report-path=-
```

# Appendix B — Getting Started

The first commands to run when beginning implementation.

```
# environment
uv init droiddesk && cd droiddesk
```

```
uv add pyside6 androguard pyaxmlparser xxhash watchdog send2trash
uv add platformdirs pydantic python-dotenv regex jinja2 weasyprint loguru
uv add sqlalchemy pyahocorasick
uv add --dev pytest pytest-qt pytest-cov mypy ruff pre-commit nuitka

# verify the toolchain is reachable
adb version
aapt2 version
gitleaks version

# confirm androguard works before building anything around it
python -c "from androguard.core.apk import APK; a = APK('sample.apk'); \
print(a.get_package(), a.get_androidversion_name(), a.get_permissions())"

# quality gates
uv run ruff check .
uv run mypy src --strict
uv run pytest
```

*Begin with Phase 1. The single most important early decision is that ProcessRunner and AdbClient are correct, cancellable, and non-blocking, because every subsequent feature is built on them.*